Homework: LLM and its Application

Complete Solutions (Exam Answer Format)

Contents

1 Q1. BERT vs GPT vs T5, and Parameter Count of a Decoder-only Transformer 3
1.1 Part 1: Comparison of BERT, GPT and T5 . . . 3
1.2 Part 2: Numerical . . . 4

2 Q2. Emergent Abilities and Data Scaling Laws 6
2.1 Part 1: Emergent abilities . . . 6
2.2 Part 2: Numerical . . . 6

3 Q3. LLM Training Pipeline, Llama 2 Architecture, and Memory Requirements 8
3.1 Part 1: Complete training pipeline of a modern LLM . . . 8
3.2 Part 2: Architectural modifications in Llama 2 . . . 9
3.3 Part 3: Numerical . . . 9

4 Q4. RLHF Framework and Bradley–Terry Preference Model 11
4.1 Part 1: RLHF framework . . . 11
4.2 Part 2: Numerical . . . 12

5 Q5. Chain-of-Thought (CoT) Prompting 14
5.1 Part 1: Chain-of-Thought prompting . . . 14
5.2 Part 2: Application to the given problem . . . 14

6 Q6. Low-Rank Adaptation (LoRA) 17
6.1 Part 1: Principle of LoRA . . . 17
6.2 Part 2: Numerical . . . 18

7 Q7. Perplexity 19
7.1 Part 1: Definition . . . 19
7.2 Part 2: Numerical . . . 19

8 Q8. Multimodal LLMs and CLIP 21
8.1 Part 1: Multimodal LLMs and cross-modal learning . . . 21
8.2 Part 2: How CLIP integrates image and text representations . . . 21
8.3 Part 3: Application to the given query . . . 22

9 Q9. GPT-3, In-Context Learning and Instruction Tuning 24
9.1 Part 1: Evolution and major developments in GPT-3 . . . 24

9.2 Part 2: Why GPT-3 performs tasks without task-specific fine-tuning 24
9.3 Part 3: Why instruction tuning improves instruction following 25

10 Q10. Deploying an LLM for a Customer-Support Chatbot 26
10.1 (a) Four challenges in deploying the LLM 26
10.2 (b) Trade-off between response quality, inference latency and computational cost . . 26
10.3 (c) Three measures to improve efficiency and reliability 27

Q1. BERT vs GPT vs T5, and Parameter Count of a Decoder-only Transformer

Compare BERT, GPT, and T5 in terms of architecture, pre-training objectives, and typical applications. Consider a decoder-only Transformer with $n = 24$ layers, hidden dimension $d = 2048$, $h = 16$ attention heads, feed-forward dimension $d_{ff} = 8192$, and parameters per layer $P_{\text{layer}} = 12d^2$. (a) Parameters in one layer. (b) Parameters in all 24 layers. (c) Embedding parameters for vocabulary 50,000 and embedding dimension 2048. (d) Total parameters including embeddings.

Part 1: Comparison of BERT, GPT and T5

All three are built from the Transformer block (multi-head self-attention + position-wise feed-forward network + residual connections + layer normalization), but they use different parts of the original encoder–decoder Transformer and are trained with different objectives.

1. BERT (Bidirectional Encoder Representations from Transformers)

- **Architecture:** Encoder-only stack. Self-attention is *bidirectional*: every token attends to all tokens to its left and right. BERT-Base: 12 layers, $d = 768$, 12 heads, 110M parameters; BERT-Large: 24 layers, $d = 1024$, 16 heads, 340M parameters.
- **Pre-training objectives:**
  (i) *Masked Language Modelling (MLM):* 15% of input tokens are selected; of these 80% are replaced by `[MASK]`, 10% by a random token and 10% left unchanged. The model predicts the original tokens: $\mathcal{L}_{\text{MLM}} = -\sum_{i \in \mathcal{M}} \log P(x_i \mid \tilde{x})$, where $\mathcal{M}$ is the set of masked positions and $\tilde{x}$ is the corrupted input.
  (ii) *Next Sentence Prediction (NSP):* binary classification of whether sentence B actually follows sentence A.
- **Typical applications:** Natural language *understanding* tasks – text classification, sentiment analysis, named entity recognition, extractive question answering (SQuAD), semantic similarity, embeddings for search/retrieval. It is not naturally suited for free-form text generation.

2. GPT (Generative Pre-trained Transformer)

- **Architecture:** Decoder-only stack with *causal (masked) self-attention*: token t can attend only to tokens $1, \ldots, t$. There is no encoder and no cross-attention.
- **Pre-training objective:** Causal (autoregressive) language modelling – predict the next token: $\mathcal{L}_{\text{CLM}} = -\sum_{t=1}^{T} \log P(x_t \mid x_1, x_2, \ldots, x_{t-1})$.
- **Typical applications:** Natural language *generation* – chatbots, story/essay writing, code generation, summarization, translation, and (from GPT-3 onwards) zero-shot and few-shot in-context learning of many tasks through prompting.

3. T5 (Text-to-Text Transfer Transformer)

- **Architecture:** Full encoder–decoder Transformer. The encoder is bidirectional; the decoder is causal and also uses cross-attention over the encoder outputs. Uses relative position biases.
- **Pre-training objective:** *Span corruption* (a denoising objective). Random contiguous spans of the input are replaced by sentinel tokens `<X>`, `<Y>`, ..., and the decoder generates the missing spans.
  Example: Input: "Thank you `<X>` me to your party `<Y>` week." Target: "`<X>` for inviting `<Y>` last `<Z>`".

- **Key idea:** Every task is cast as *text-to-text*: a task prefix is prepended to the input (e.g. "translate English to German: ...", "summarize: ...") and the output is always text.
- **Typical applications:** Translation, summarization, question answering, classification (output label as text) – any sequence-to-sequence task.

Comparison table

Aspect	BERT	GPT	T5
Architecture	Encoder-only	Decoder-only	Encoder–Decoder
Attention	Bidirectional	Causal (left-to-right)	Bidirectional (enc.), causal + cross (dec.)
Pre-training objective	MLM + NSP	Next-token prediction	Span corruption (denoising)
Output	Contextual embeddings / labels	Generated text	Generated text
Strength	Understanding	Generation, in-context learning	Unified seq2seq tasks
Typical use	Classification, NER, extractive QA	Chat, code, writing, few-shot tasks	Translation, summarization, QA
Adaptation	Fine-tune with task head	Prompting / fine-tuning	Fine-tune with task prefix

Part 2: Numerical

Given: $n = 24$, $d = 2048$, $h = 16$, $d_{ff} = 8192$, $P_{\text{layer}} = 12d^2$, $V = 50{,}000$, $d_{\text{emb}} = 2048$.

Origin of the formula $12d^2$ (for understanding):

- Attention: four projection matrices W_Q, W_K, W_V, W_O, each $d \times d \Rightarrow 4d^2$.
- Feed-forward: $W_1 \in \mathbb{R}^{d \times d_{ff}}$ and $W_2 \in \mathbb{R}^{d_{ff} \times d} \Rightarrow 2\,d\,d_{ff}$. Here $d_{ff} = 8192 = 4 \times 2048 = 4d$, so $2\,d\,(4d) = 8d^2$.
- Total: $4d^2 + 8d^2 = 12d^2$ (biases and LayerNorm parameters are negligible and ignored). The number of heads h does not change the count, since the heads only split d into h parts of size $d/h = 128$.

(a) Parameters in one Transformer layer

$$d^2 = 2048 \times 2048 = 4{,}194{,}304$$
$$P_{\text{layer}} = 12d^2 = 12 \times 4{,}194{,}304 = 50{,}331{,}648$$

$P_{\text{layer}} = 50{,}331{,}648 \approx 50.3$ million parameters.

(b) Parameters in all 24 layers

$$P_{\text{layers}} = n \times P_{\text{layer}} = 24 \times 50{,}331{,}648 = 1{,}207{,}959{,}552$$

$P_{\text{layers}} = 1{,}207{,}959{,}552 \approx 1.21$ billion parameters.

(c) Input embedding parameters

The embedding matrix has one row of size d_{emb} for every token of the vocabulary: $E \in \mathbb{R}^{V \times d_{\text{emb}}}$.

$$P_{\text{emb}} = V \times d_{\text{emb}} = 50{,}000 \times 2048 = 102{,}400{,}000$$

$P_{\text{emb}} = 102{,}400{,}000 \approx 102.4$ million parameters.

(d) Total parameter count

$$\begin{aligned} P_{\text{total}} &= P_{\text{layers}} + P_{\text{emb}} \\ &= 1{,}207{,}959{,}552 + 102{,}400{,}000 \\ &= 1{,}310{,}359{,}552 \end{aligned}$$

$P_{\text{total}} = 1{,}310{,}359{,}552 \approx 1.31$ billion parameters.

Observation: The embedding layer contributes $\dfrac{102{,}400{,}000}{1{,}310{,}359{,}552} \times 100 \approx 7.8\%$ of the total; about 92% of the parameters lie in the Transformer layers.

Q2. Emergent Abilities and Data Scaling Laws

What are emergent abilities in LLMs? Explain how scaling can lead to the emergence of new capabilities, with examples. An LLM is trained on Dataset A (100 billion tokens) and Dataset B (400 billion tokens). Loss follows $L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D}$ with $\alpha_D = 0.5$. (a) Ratio L_B/L_A. (b) L_B if $L_A = 2.0$. (c) Significance of scaling laws in planning LLM training.

Part 1: Emergent abilities

Definition: An ability is called *emergent* if it is *not present in smaller models but is present in larger models.* On a plot of task performance versus model scale (parameters, data or training compute), performance stays near random chance for small models and then rises sharply and unpredictably once the scale crosses a certain threshold. Such abilities cannot be predicted by simply extrapolating the performance of smaller models.

How scaling leads to emergence:

1. **Smooth loss, sharp task metrics.** The next-token loss decreases smoothly (as a power law) with scale. Many tasks, however, need several sub-skills to *all* be correct at once (e.g. every step of a multi-step calculation). If each step is correct with probability p, a k-step task succeeds with probability p^k. As p improves gradually, p^k stays near zero for a long time and then rises rapidly, which appears as a sudden jump.
2. **Exact-match metrics.** Metrics such as exact match give zero credit to partially correct answers, so gradual improvement is hidden until answers become fully correct.
3. **Capacity for richer representations.** Larger models trained on more diverse data store more world knowledge and learn more abstract internal representations (e.g. of arithmetic, syntax, logical structure) that can be combined to solve new tasks.
4. **Diverse data.** Very large corpora contain implicit demonstrations of many tasks (Q&A pairs, translations, worked solutions), which large models can absorb and later reuse.

Examples of emergent abilities:

- **In-context (few-shot) learning:** GPT-3 (175B) can learn a new task from a few examples in the prompt without weight updates; much smaller GPT models could not do this reliably.
- **Chain-of-thought reasoning:** Prompting with step-by-step reasoning improves multi-step arithmetic (e.g. GSM8K) only for models of roughly 60–100B parameters and above; for small models it does not help or even hurts.
- **Multi-digit arithmetic:** 3-digit addition/subtraction accuracy jumps from near zero to high accuracy at large scale.
- **Instruction following:** Large instruction-tuned models generalize to unseen instructions, while small models do not.
- **Others:** word unscrambling, transliteration, code generation from natural language descriptions, multi-task language understanding (MMLU) above chance.

Part 2: Numerical

Given: $D_A = 100 \times 10^9$ tokens, $D_B = 400 \times 10^9$ tokens, $\alpha_D = 0.5$, $L(D) = (D_c/D)^{\alpha_D}$, $L_A = 2.0$.

(a) Ratio of loss for Dataset B to Dataset A

$$\frac{L_B}{L_A} = \frac{\left(\frac{D_c}{D_B}\right)^{\alpha_D}}{\left(\frac{D_c}{D_A}\right)^{\alpha_D}} = \left(\frac{D_c}{D_B} \cdot \frac{D_A}{D_c}\right)^{\alpha_D} = \left(\frac{D_A}{D_B}\right)^{\alpha_D}$$

The constant D_c cancels. Substituting values:

$$\frac{L_B}{L_A} = \left(\frac{100 \times 10^9}{400 \times 10^9}\right)^{0.5} = \left(\frac{1}{4}\right)^{0.5} = \frac{1}{\sqrt{4}} = \frac{1}{2} = 0.5$$

$\frac{L_B}{L_A} = 0.5$, i.e. increasing the data 4× halves the loss.

(b) Loss for Dataset B

$$L_B = 0.5 \times L_A = 0.5 \times 2.0 = 1.0$$

$L_B = 1.0$ (a reduction of 1.0, i.e. 50%, compared with $L_A = 2.0$).

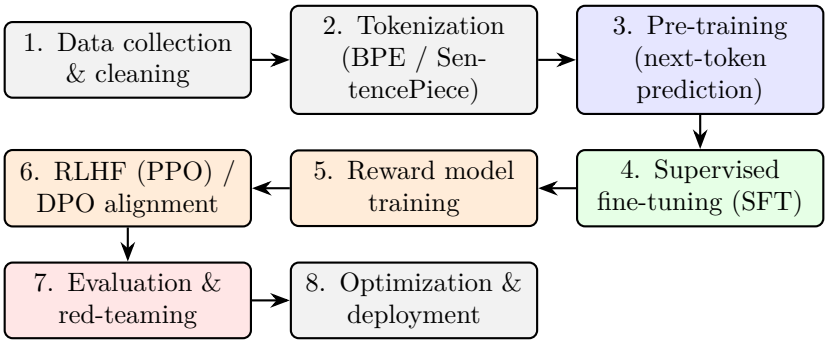
(c) Significance of scaling laws in planning LLM training

1. **Predictability:** Scaling laws express loss as a power law in model size N, data size D and compute C, e.g. $L(N) = (N_c/N)^{\alpha_N}$, $L(D) = (D_c/D)^{\alpha_D}$. Small, cheap experiments can be fitted and extrapolated to predict the loss of a very large run *before* spending millions on it.
2. **Optimal allocation of compute:** For a fixed compute budget ($C \approx 6ND$ FLOPs), scaling laws decide how to split the budget between model size and data. The Chinchilla study (Hoffmann et al., 2022) showed that N and D should be scaled roughly equally, about 20 tokens per parameter; earlier models such as GPT-3 were under-trained on data.
3. **Data requirement estimation:** As in parts (a)–(b), they tell us how much additional data is needed to reach a target loss (here 4× data ⇒ loss halves when $\alpha_D = 0.5$).
4. **Diminishing returns:** Because of the power law, each further reduction in loss needs a multiplicative increase in resources. This helps decide when further scaling is no longer cost-effective.
5. **Hardware and budget planning:** Estimates of GPU count, training time, cost and energy can be made in advance.
6. **Hyper-parameter and architecture decisions:** Learning rate, batch size and depth/width choices can be tuned at small scale and transferred to large scale.
7. **Risk reduction:** Reduces the chance of a failed or wasteful large training run.

Q3. LLM Training Pipeline, Llama 2 Architecture, and Memory Requirements

Explain the complete training pipeline of a modern LLM. Explain the major architectural modifications introduced in Llama 2 compared with the conventional Transformer. Llama 2 comes in 7B, 13B and 70B configurations; FP16 precision. (a) Memory to store each model. (b) Additional memory for 70B vs 7B. (c) Which models fit on a 24 GB GPU (parameter storage only)? (d) Why is actual GPU memory higher than parameter storage?

Part 1: Complete training pipeline of a modern LLM



1. **Data collection and pre-processing:** Trillions of tokens are gathered from web crawls (e.g. Common Crawl), books, Wikipedia, scientific papers, and code. The data is cleaned: de-duplication, language identification, quality filtering, removal of toxic content and personal information, and mixing of sources in suitable proportions.

2. **Tokenization:** Text is split into sub-word tokens using Byte-Pair Encoding (BPE) or SentencePiece and mapped to integer IDs from a fixed vocabulary (Llama 2 uses a 32,000-token SentencePiece vocabulary).

3. **Pre-training (self-supervised):** A decoder-only Transformer is trained to predict the next token by minimizing the cross-entropy loss $\mathcal{L} = -\sum_t \log P_\theta(x_t \mid x_{<t})$. This is the most expensive stage (thousands of GPUs for weeks) and uses data, tensor and pipeline parallelism, mixed precision, and the AdamW optimizer with learning-rate warm-up and cosine decay. The result is a *base model* that has broad knowledge but does not reliably follow instructions.

4. **Supervised Fine-Tuning (SFT) / instruction tuning:** The base model is fine-tuned on high-quality (prompt, response) pairs written by humans, so that it learns to follow instructions and conversational format. The loss is computed only on response tokens.

5. **Reward model training:** For each prompt, multiple model responses are ranked by human annotators. A reward model $r_\phi(x, y)$ is trained so that preferred responses get higher scores (Bradley–Terry loss, see Q4).

6. **Reinforcement Learning from Human Feedback (RLHF):** The SFT model (policy) is optimized with PPO to maximize the reward while a KL-divergence penalty keeps it close to the SFT model. Alternatives: rejection-sampling fine-tuning (used in Llama 2 together with PPO) or Direct Preference Optimization (DPO).

7. **Evaluation and safety testing:** Benchmarks (MMLU, GSM8K, HumanEval, TruthfulQA, etc.), human evaluation of helpfulness and harmlessness, and red-teaming for safety.

8. **Optimization and deployment:** Quantization (INT8/INT4), distillation, KV-cache and batching optimizations, serving behind an API, followed by continuous monitoring and feedback collection.

Part 2: Architectural modifications in Llama 2

Llama 2 is a decoder-only Transformer, but it differs from the original Transformer (Vaswani et al., 2017) in the following ways:

Component	Conventional Transformer	Llama 2
Normalization position	Post-normalization: LayerNorm *after* the residual addition	Pre-normalization: normalize the *input* of each sub-layer, which gives more stable training of deep networks
Normalization type	LayerNorm (mean and variance, with bias)	RMSNorm: $\mathrm{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \odot g$ (no mean subtraction, cheaper)
Activation in FFN	ReLU: $\mathrm{FFN}(x) = \max(0, xW_1)W_2$	SwiGLU: $\mathrm{FFN}(x) = \big(\mathrm{Swish}(xW_1) \odot xW_3\big)W_2$, where $\mathrm{Swish}(z) = z\,\sigma(z)$; FFN width $\approx \frac{2}{3} \cdot 4d$
Positional information	Absolute sinusoidal embeddings added to the input	Rotary Positional Embeddings (RoPE): Q and K vectors are rotated by position-dependent angles in each layer, encoding *relative* position
Attention	Multi-Head Attention (each head has its own K, V)	Grouped-Query Attention (GQA) in the 34B and 70B models: groups of query heads share one K/V head, which shrinks the KV cache and speeds up inference
Context length	512 tokens (original)	4096 tokens (twice that of Llama 1)
Training data	–	2 trillion tokens (40% more than Llama 1)
Bias terms	Present in linear layers	Removed from linear layers
Chat variant	–	Llama 2-Chat: SFT + RLHF (rejection sampling + PPO), Ghost Attention (GAtt) for multi-turn consistency

Part 3: Numerical

Given: Model sizes 7×10^9, 13×10^9, 70×10^9 parameters. FP16 (16-bit floating point) uses 16 bits = 2 bytes per parameter. GPU memory = 24 GB.
Formula: Memory = Number of parameters × Bytes per parameter, with 1 GB = 10^9 bytes.

(a) Memory to store each model

$$M_{7\mathrm{B}} = 7 \times 10^9 \times 2 \text{ bytes} = 14 \times 10^9 \text{ bytes} = 14 \text{ GB}$$
$$M_{13\mathrm{B}} = 13 \times 10^9 \times 2 \text{ bytes} = 26 \times 10^9 \text{ bytes} = 26 \text{ GB}$$
$$M_{70\mathrm{B}} = 70 \times 10^9 \times 2 \text{ bytes} = 140 \times 10^9 \text{ bytes} = 140 \text{ GB}$$

7B: 14 GB 13B: 26 GB 70B: 140 GB
(In binary units, 1 GiB = 2^{30} bytes: 13.04 GiB, 24.21 GiB and 130.39 GiB respectively.)

(b) Additional memory for 70B compared with 7B

$$\Delta M = M_{70\mathrm{B}} - M_{7\mathrm{B}} = 140 - 14 = 126 \text{ GB}$$
$$\text{(equivalently)} \quad \Delta M = (70 - 7) \times 10^9 \times 2 = 63 \times 10^9 \times 2 = 126 \text{ GB}$$

The 70B model needs 126 GB more, i.e. $\frac{140}{14} = 10$ times the memory of the 7B model.

(c) Which models fit on a 24 GB GPU (parameter storage only)?

Model	Memory (FP16)	Compare with 24 GB	Fits?
7B	14 GB	$14 < 24$	Yes (10 GB left)
13B	26 GB	$26 > 24$	No (2 GB short)
70B	140 GB	$140 > 24$	No (116 GB short)

Only the **Llama 2 7B** model can theoretically fit on a 24 GB GPU in FP16. The 13B and 70B models would need quantization (e.g. INT8/INT4) or multiple GPUs.

(d) Why actual GPU memory consumption is higher than parameter storage

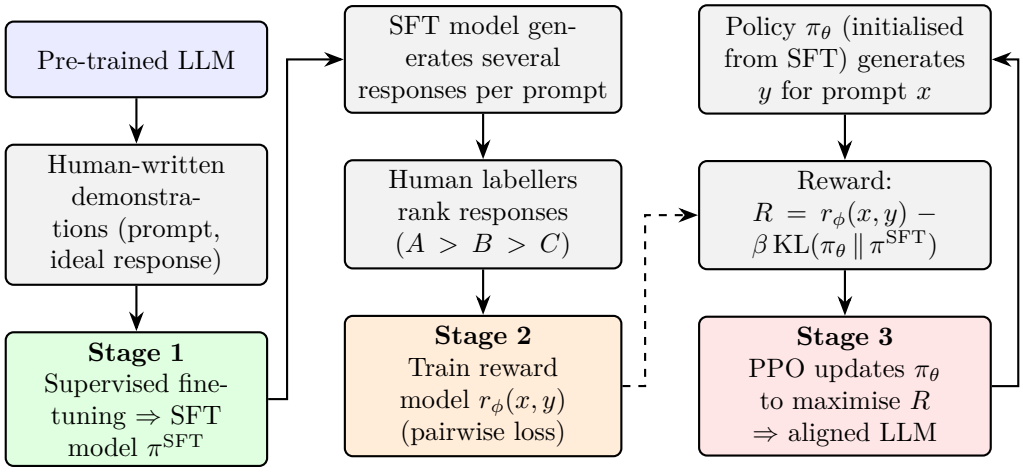
1. **Activations:** Intermediate outputs of every layer (attention scores, FFN outputs) must be held in memory during the forward pass; in training they are stored for back-propagation. They grow with batch size and sequence length; attention scores grow as $O(n^2)$ in sequence length n.
2. **KV cache (inference):** During autoregressive generation, keys and values of all previous tokens are cached for every layer. Size $\approx 2 \times n_{\text{layers}} \times n_{\text{tokens}} \times d \times \text{batch} \times \text{bytes}$; for long contexts and large batches this can be several GB.
3. **Gradients (training):** One gradient per parameter, i.e. the same size as the weights again (2 bytes/parameter in FP16).
4. **Optimizer states (training):** Adam stores first and second moments, usually in FP32, plus an FP32 master copy of weights: about $4 + 4 + 4 = 12$ extra bytes per parameter. Full mixed-precision training therefore needs about 16 bytes/parameter (e.g. $7\text{B} \times 16 \approx 112$ GB).
5. **Framework and CUDA overhead:** CUDA context, cuBLAS/cuDNN kernels and workspaces, typically several hundred MB to 1–2 GB.
6. **Memory fragmentation:** The memory allocator reserves blocks that are not fully used.
7. **Temporary buffers and input batches:** Token IDs, embeddings, logits over the vocabulary (batch $\times\, n \times V$), and communication buffers in multi-GPU setups.

Q4. RLHF Framework and Bradley–Terry Preference Model

Explain the RLHF framework with a block diagram, and why RLHF is important for alignment. Human ranking: $A > B > C$. Reward model scores: $r(A) = 2.5$, $r(B) = 1.5$, $r(C) = 0.5$. Bradley–Terry: $P(A > B) = \dfrac{e^{r(A)}}{e^{r(A)} + e^{r(B)}}$. (a) $P(A > B)$. (b) $P(B > C)$. (c) $P(A > C)$. (d) How a reward model learns human preferences from ranked responses.

Part 1: RLHF framework

Reinforcement Learning from Human Feedback (RLHF) is a method of fine-tuning a language model so that its outputs match human preferences (helpful, honest, harmless). It has three stages.



Stage 1 – Supervised Fine-Tuning (SFT): The pre-trained model is fine-tuned on a dataset of prompts with high-quality human-written responses, giving the model π^{SFT}, which can follow instructions in the desired format.

Stage 2 – Reward Model (RM) training: For each prompt x, the SFT model produces K responses (e.g. $K = 4$ to 9). Human labellers rank them. A reward model $r_\phi(x, y)$ – usually the SFT model with its final layer replaced by a linear layer that outputs a single scalar – is trained so that preferred responses get higher scores, using the pairwise loss

$$\mathcal{L}_{\text{RM}}(\phi) = -\,\mathbb{E}_{(x,y_w,y_l)}\Big[\log \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)\Big],$$

where y_w is the preferred (winning) and y_l the less preferred (losing) response and $\sigma(z) = \dfrac{1}{1 + e^{-z}}$.

Stage 3 – RL fine-tuning with PPO: The policy π_θ (initialised from π^{SFT}) generates responses; the reward model scores them; Proximal Policy Optimization (PPO) updates π_θ to maximise

$$J(\theta) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\Big[r_\phi(x, y)\Big] - \beta\,\mathrm{KL}\Big(\pi_\theta(\cdot \mid x) \,\|\, \pi^{\text{SFT}}(\cdot \mid x)\Big).$$

The KL penalty (weighted by β) prevents the policy from drifting too far from the SFT model, which avoids *reward hacking* (producing text that fools the reward model) and preserves fluency.

Why RLHF is important for alignment:

1. **Objective mismatch:** Pre-training only maximizes likelihood of internet text; it does not ask for helpful, truthful or safe answers. RLHF optimizes directly for what humans prefer.

2. **Hard-to-specify goals:** Qualities like "helpful", "polite" or "not harmful" are difficult to write as a loss function, but humans can easily *compare* two answers. RLHF turns these comparisons into a learnable reward.
3. **Safety:** Reduces toxic, biased and dangerous outputs and teaches the model to refuse harmful requests.
4. **Honesty:** Reduces hallucination and encourages the model to express uncertainty.
5. **Better instruction following:** InstructGPT (1.3B) trained with RLHF was preferred by humans over the 175B GPT-3, showing alignment can matter more than size.
6. **Learning from the whole response:** Unlike token-level SFT, the reward evaluates the complete response, so the model learns sequence-level qualities such as coherence and usefulness.

Part 2: Numerical

Given: $r(A) = 2.5$, $r(B) = 1.5$, $r(C) = 0.5$.

Formula: $P(i > j) = \dfrac{e^{r(i)}}{e^{r(i)} + e^{r(j)}}$. Dividing numerator and denominator by $e^{r(i)}$:

$$P(i > j) = \frac{1}{1 + e^{-(r(i)-r(j))}} = \sigma(r(i) - r(j)).$$

So the probability depends only on the *difference* of rewards.

Exponential values needed:

$$e^{2.5} = 12.1825, \qquad e^{1.5} = 4.4817, \qquad e^{0.5} = 1.6487, \qquad e^{-1} = 0.3679, \qquad e^{-2} = 0.1353.$$

(a) $P(A > B)$

$$P(A > B) = \frac{e^{2.5}}{e^{2.5} + e^{1.5}} = \frac{12.1825}{12.1825 + 4.4817} = \frac{12.1825}{16.6642} = 0.7311$$

$$\text{Check:} \quad P(A > B) = \frac{1}{1 + e^{-(2.5-1.5)}} = \frac{1}{1 + e^{-1}} = \frac{1}{1.3679} = 0.7311$$

$P(A > B) \approx 0.731$ (73.1%)

(b) $P(B > C)$

$$P(B > C) = \frac{e^{1.5}}{e^{1.5} + e^{0.5}} = \frac{4.4817}{4.4817 + 1.6487} = \frac{4.4817}{6.1304} = 0.7311$$

$$\text{Check:} \quad P(B > C) = \frac{1}{1 + e^{-(1.5-0.5)}} = \frac{1}{1 + e^{-1}} = 0.7311$$

$P(B > C) \approx 0.731$ (73.1%) – same as (a) because the reward difference is again 1.0.

(c) $P(A > C)$

$$P(A > C) = \frac{e^{2.5}}{e^{2.5} + e^{0.5}} = \frac{12.1825}{12.1825 + 1.6487} = \frac{12.1825}{13.8312} = 0.8808$$

$$\text{Check:} \quad P(A > C) = \frac{1}{1 + e^{-(2.5-0.5)}} = \frac{1}{1 + e^{-2}} = \frac{1}{1.1353} = 0.8808$$

$P(A > C) \approx 0.881$ (88.1%)

Interpretation: All three probabilities are above 0.5 in the direction of the human ranking $A > B > C$, so the reward model agrees with the human evaluators. The largest reward gap (A vs C, gap = 2) gives the highest confidence (0.881), while adjacent pairs (gap = 1) give 0.731.

(d) How a reward model learns human preferences from ranked responses

1. **Collect rankings:** For a prompt x, K responses are ranked by humans. A ranking of K responses gives $\binom{K}{2}$ pairwise comparisons. For $A > B > C$ ($K = 3$): $\binom{3}{2} = 3$ pairs: (A, B), (B, C), (A, C), each written as (winner y_w, loser y_l).

2. **Score each response:** The reward model outputs a scalar $r_\phi(x, y)$ for each response.

3. **Model preference probability:** Using Bradley–Terry, $P(y_w > y_l) = \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)$.

4. **Loss:** Maximize the likelihood of the human choices, i.e. minimize the negative log-likelihood averaged over pairs:
$$\mathcal{L}(\phi) = -\frac{1}{\binom{K}{2}} \sum_{(y_w, y_l)} \log \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big).$$
For the given scores: $-\ln(0.7311) = 0.3133$ for (A, B), 0.3133 for (B, C), and $-\ln(0.8808) = 0.1269$ for (A, C). Average loss $= \dfrac{0.3133 + 0.3133 + 0.1269}{3} = \dfrac{0.7535}{3} = 0.2512$.

5. **Gradient update:** The gradient pushes $r_\phi(x, y_w)$ up and $r_\phi(x, y_l)$ down. The size of the update is proportional to $1 - \sigma(\Delta r)$, so pairs the model already ranks confidently contribute little, while wrongly ranked pairs contribute a lot.

6. **Generalization:** Trained over many thousands of prompts, the model learns general features of good answers (correctness, relevance, clarity, safety), so it can score new, unseen responses. Only reward *differences* matter, so the absolute scale of rewards is arbitrary (often normalized).

Q5. Chain-of-Thought (CoT) Prompting

What is CoT prompting? How does breaking a problem into intermediate steps improve reasoning? Problem: A company has 240 employees; 35% are in research; of the remaining, 40% are in development; the rest are in administration. Find the number in each department. (a) Direct prompting. (b) Formulate a CoT prompt. (c) Show the reasoning steps and calculate. (d) Compare direct and CoT prompting.

Part 1: Chain-of-Thought prompting

Definition: Chain-of-Thought (CoT) prompting is a prompting technique in which the LLM is encouraged to generate a sequence of *intermediate natural-language reasoning steps* before giving the final answer (Wei et al., 2022). It can be done in two ways:

- **Few-shot CoT:** The prompt contains a few example questions whose answers include worked-out step-by-step reasoning.
- **Zero-shot CoT:** A trigger phrase such as "*Let's think step by step*" is added to the question (Kojima et al., 2022).

Why intermediate steps improve performance:

1. **Decomposition:** A complex problem is split into simpler sub-problems, each of which the model can solve reliably.
2. **More computation per answer:** An LLM spends a fixed amount of computation per generated token. Writing out the reasoning gives the model more tokens, hence more computation, before it commits to an answer.
3. **Working memory:** Intermediate results (e.g. 84, 156) are written into the context, so later steps can attend to them instead of holding everything implicitly in hidden states.
4. **Matches training data:** Text such as textbooks and worked solutions contains step-by-step explanations, so this style triggers patterns the model has already learned.
5. **Error reduction and checking:** Each step is conditioned on correct earlier steps, which reduces compounding errors, and the visible steps can be checked.
6. CoT is most beneficial for arithmetic word problems, commonsense and symbolic reasoning, and multi-hop questions; the benefit is an emergent ability that appears mainly in large models (roughly 100B parameters and above).

Part 2: Application to the given problem

(a) Direct prompting

In direct (standard) prompting, the question is given and the model is asked for *only* the final answer, with no intermediate reasoning.

Example direct prompt:

```
Q: A company has 240 employees.  Of these, 35% are working in the research
department.  Among the remaining employees, 40% are working in the development
department.  The rest are working in administration.  Give only the final number
of employees in each department.
A:
```

The model must map the question to the answer in a single step, internally computing 35% of 240, the remainder, 40% of the *remainder* (not of 240), and the rest. Typical errors in direct mode are:

computing 40% of the total 240 (giving 96) instead of the remaining employees, or subtracting the percentages ($100\% - 35\% - 40\% = 25\%$) and wrongly giving administration as 60. Because no steps are shown, such an error cannot be traced.

(b) Chain-of-Thought prompt

Few-shot CoT prompt (one worked example + the target question):

```
Q: A school has 200 students.  30% play football.  Of the remaining students, 50%
play cricket.  The rest play no sport.  How many students are in each group?
A: Let's solve step by step.
Step 1:  Football = 30% of 200 = 0.30 x 200 = 60.
Step 2:  Remaining = 200 - 60 = 140.
Step 3:  Cricket = 50% of 140 = 0.50 x 140 = 70.
Step 4:  No sport = 140 - 70 = 70.
Step 5:  Check:  60 + 70 + 70 = 200.  Correct.
Final answer:  Football = 60, Cricket = 70, No sport = 70.

Q: A company has 240 employees.  Of these, 35% are working in the research
department.  Among the remaining employees, 40% are working in the development
department.  The rest are working in administration.  Calculate the number of
employees in each department.
A: Let's solve step by step.
```

Zero-shot CoT alternative: "`[question]` Let's think step by step. First find the research employees, then the remaining employees, then development as 40% of the remaining, then administration as what is left. Verify that the three totals add to 240, then state the final answer."

(c) Expected reasoning steps and calculation

Given: Total $T = 240$; Research = 35% of T; Development = 40% of (remaining after research); Administration = rest.

Step 1 – Research department:

$$R = 35\% \times 240 = \frac{35}{100} \times 240 = 0.35 \times 240 = 84$$

Step 2 – Remaining employees:

$$\text{Remaining} = 240 - 84 = 156 \qquad (\text{or } 65\% \times 240 = 0.65 \times 240 = 156)$$

Step 3 – Development department (40% of the remaining):

$$D = 40\% \times 156 = 0.40 \times 156 = 62.4$$

Step 4 – Administration (the rest of the remaining):

$$A = 156 - 62.4 = 93.6 \qquad (\text{or } 60\% \times 156 = 0.60 \times 156 = 93.6)$$

Step 5 – Verification:

$$R + D + A = 84 + 62.4 + 93.6 = 240 \quad \checkmark$$

As fractions of the total: Research 35%, Development $0.65 \times 0.40 = 26\%$, Administration $0.65 \times 0.60 = 39\%$; $35 + 26 + 39 = 100\%$ ✓.

Step 6 – Whole-number check: Since the number of employees must be a whole number, the exact values 62.4 and 93.6 cannot be literal head-counts. Rounding to the nearest whole number while keeping the total at 240 gives Development = 62 and Administration = 94 ($84+62+94 = 240$). A good CoT response exposes this issue in the data, which a direct answer would hide.

Research:	84 employees	(35%)
Development:	$62.4 \approx 62$ employees	(26%)
Administration:	$93.6 \approx 94$ employees	(39%)
Total:	240 employees	

(d) Comparison of Direct prompting and CoT prompting

Criterion	Direct Prompting	Chain-of-Thought Prompting
Reasoning transparency	Low: only the final answer is shown; the reasoning is a "black box".	High: every intermediate step (84, 156, 62.4, 93.6) is visible and can be read and understood.
Suitability for multi-step problems	Poor: the model must do all steps implicitly in one pass; errors such as taking 40% of 240 instead of 156 are common.	Good: decomposes the problem; each step is simple; much higher accuracy on arithmetic, logic and multi-hop tasks.
Answer verification	Difficult: the answer can only be checked by solving the problem again independently.	Easy: each step can be checked, the wrong step can be located, and a sum check ($84 + 62.4 + 93.6 = 240$) is built in. Also allows self-consistency (majority vote over several reasoning paths).
Output length / cost	Short output; fast, fewer tokens, lower cost.	Longer output; more tokens, higher latency and cost.
Best for	Simple factual or single-step questions.	Arithmetic word problems, logical and commonsense reasoning, planning.
Model size	Works for all sizes.	Benefit appears mainly in large models; small models may produce fluent but wrong chains.
Risk	Silent errors.	Plausible-looking reasoning can still contain an error, so steps must be checked.

Q6. Low-Rank Adaptation (LoRA)

Explain LoRA with mathematical formulation and diagram. State two advantages over full fine-tuning. $W \in \mathbb{R}^{4096\times4096}$, rank $r = 16$, $W' = W + \frac{\alpha}{r}BA$ with $A \in \mathbb{R}^{r\times4096}$, $B \in \mathbb{R}^{4096\times r}$. (a) Parameters in W. (b) Trainable parameters in A and B. (c) Percentage reduction vs full fine-tuning. (d) Scaling factor for $\alpha = 32$.

Part 1: Principle of LoRA

Idea: When a large pre-trained model is fine-tuned, the change in weights ΔW has a low "intrinsic rank": the adaptation lies in a low-dimensional subspace. LoRA (Hu et al., 2021) therefore *freezes* the pre-trained weight W_0 and learns the update as a product of two small matrices.

Mathematical formulation: For a pre-trained weight $W_0 \in \mathbb{R}^{d\times k}$, full fine-tuning learns $W = W_0 + \Delta W$ with $\Delta W \in \mathbb{R}^{d\times k}$ ($d \cdot k$ parameters). LoRA restricts

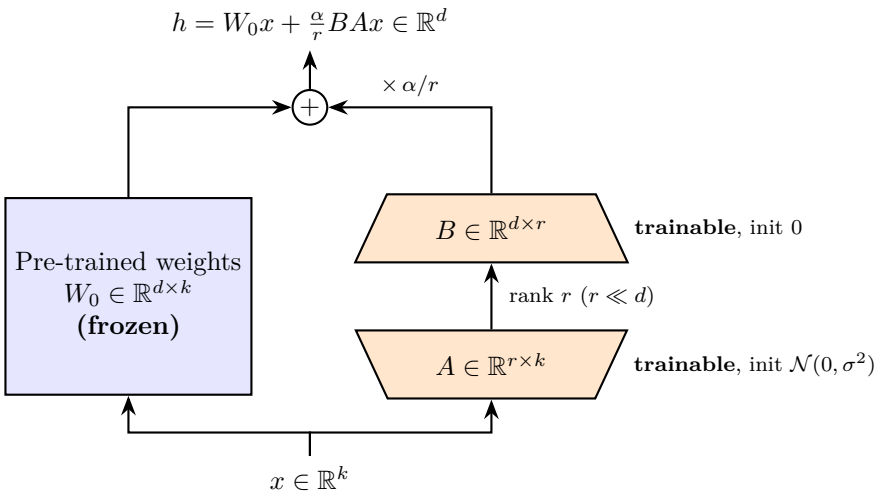
$$\Delta W = \frac{\alpha}{r} BA, \qquad B \in \mathbb{R}^{d\times r}, \quad A \in \mathbb{R}^{r\times k}, \quad r \ll \min(d, k).$$

The forward pass for input x becomes

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} B(Ax).$$

- W_0 is frozen (no gradients); only A and B are trained.
- Number of trainable parameters $= r(d + k)$ instead of dk.
- $\text{rank}(BA) \le r$.
- **Initialization:** A is initialized with small random Gaussian values and $B = 0$, so $BA = 0$ at the start; the model initially behaves exactly like the pre-trained model.
- **Scaling:** α/r controls the magnitude of the update and reduces the need to retune the learning rate when r changes.
- LoRA is usually applied to the attention projections W_Q, W_K, W_V, W_O (and optionally FFN weights).
- **Merging for inference:** After training, $W' = W_0 + \frac{\alpha}{r}BA$ is computed once, so inference has no extra latency.

Diagram:



Advantages of LoRA over full fine-tuning (any two required):

1. **Far fewer trainable parameters and less memory:** Gradients and optimizer states are needed only for A and B (less than 1% of the weights), so GPU memory for training drops by up to about 3× and large models can be fine-tuned on a single GPU.
2. **Small storage and easy task switching:** Each task needs to store only the small A, B matrices (a few MB) instead of a full model copy (many GB). One frozen base model can serve many tasks by swapping adapters.
3. **No extra inference latency:** BA can be merged into W_0 after training, unlike adapter layers that add extra computation.
4. **Less catastrophic forgetting and over-fitting:** The original weights stay unchanged, preserving general knowledge, and the low-rank constraint acts as a regularizer on small datasets.
5. **Comparable quality:** On many benchmarks LoRA matches the accuracy of full fine-tuning.

Part 2: Numerical

Given: $d = k = 4096$, $r = 16$, $\alpha = 32$, $A \in \mathbb{R}^{16\times 4096}$, $B \in \mathbb{R}^{4096\times 16}$.

(a) Parameters in the original weight matrix

$$|W| = d \times k = 4096 \times 4096 = 16{,}777{,}216$$

$|W| = 16{,}777{,}216 \approx 16.78$ million parameters (all trainable in full fine-tuning).

(b) Trainable parameters in A and B

$$\begin{aligned} |A| &= r \times k = 16 \times 4096 = 65{,}536 \\ |B| &= d \times r = 4096 \times 16 = 65{,}536 \\ |A| + |B| &= 65{,}536 + 65{,}536 = 131{,}072 \qquad [= r(d+k) = 16 \times 8192] \end{aligned}$$

Trainable parameters with LoRA = 131,072.

(c) Percentage reduction in trainable parameters

$$\text{Fraction trained} = \frac{131{,}072}{16{,}777{,}216} = \frac{2 \times 16 \times 4096}{4096 \times 4096} = \frac{32}{4096} = \frac{1}{128} = 0.0078125 = 0.78125\%$$

$$\begin{aligned} \text{Reduction} &= \frac{16{,}777{,}216 - 131{,}072}{16{,}777{,}216} \times 100\% = \frac{16{,}646{,}144}{16{,}777{,}216} \times 100\% \\ &= (1 - 0.0078125) \times 100\% = 99.21875\% \end{aligned}$$

Reduction = 99.22%. LoRA trains only 0.78% of the parameters, i.e. 128× fewer than full fine-tuning.

(d) Scaling factor for $\alpha = 32$

$$\text{Scaling factor} = \frac{\alpha}{r} = \frac{32}{16} = 2$$

Scaling factor = 2, so $W' = W + 2\,BA$: the low-rank update is doubled before it is added to W.

Q7. Perplexity

Define perplexity and explain how it measures predictive performance. Two models are evaluated on the same test sequence of 100 tokens. Total negative log-likelihood (NLL): Model A = 120, Model B = 90. (a) Average cross-entropy loss. (b) Perplexity. (c) Which model is better? Justify.

Part 1: Definition

For a test sequence $W = (w_1, w_2, \ldots, w_N)$, **perplexity (PPL)** is the inverse probability of the sequence under the model, normalized by the number of tokens:

$$\mathrm{PPL}(W) = P(w_1, w_2, \ldots, w_N)^{-\frac{1}{N}} = \left(\prod_{i=1}^{N} \frac{1}{P(w_i \mid w_1, \ldots, w_{i-1})} \right)^{\frac{1}{N}}.$$

Taking logarithms, it is the exponential of the average cross-entropy (average negative log-likelihood per token):

$$H = -\frac{1}{N} \sum_{i=1}^{N} \ln P(w_i \mid w_{<i}) = \frac{\mathrm{NLL}_{\mathrm{total}}}{N}, \qquad \mathrm{PPL} = e^{H}.$$

(With natural logarithms the base is e; if base-2 logarithms are used, $\mathrm{PPL} = 2^{H}$.)

How it measures predictive performance:

- **Interpretation as branching factor:** A perplexity of k means the model is, on average, as uncertain as if it were choosing uniformly among k equally likely tokens at each step.
- **Lower is better:** Higher probability assigned to the actual next tokens ⇒ lower cross-entropy ⇒ lower perplexity. Minimum value is 1 (perfect prediction, $P = 1$ for every token). A uniform model over vocabulary V has $\mathrm{PPL} = |V|$.
- **Normalization:** Dividing by N makes the score independent of the sequence length, so different sequences can be compared.
- **Limitations:** Comparisons are valid only on the same test set with the same tokenizer/vocabulary; low perplexity does not guarantee factual correctness or usefulness in downstream tasks.

Part 2: Numerical

Given: $N = 100$ tokens; $\mathrm{NLL}_A = 120$; $\mathrm{NLL}_B = 90$.
Formula: $H = \dfrac{\mathrm{NLL}_{\mathrm{total}}}{N}, \quad \mathrm{PPL} = e^{H}.$

(a) Average cross-entropy loss

$$H_A = \frac{120}{100} = 1.2 \text{ nats/token}$$
$$H_B = \frac{90}{100} = 0.9 \text{ nats/token}$$

$H_A = 1.2, \quad H_B = 0.9$

(b) Perplexity

$$\mathrm{PPL}_A = e^{H_A} = e^{1.2} = 3.320$$
$$\mathrm{PPL}_B = e^{H_B} = e^{0.9} = 2.460$$

(Step: $e^{1.2} = e^1 \times e^{0.2} = 2.7183 \times 1.2214 = 3.3201$; $e^{0.9} = e^1 \times e^{-0.1} = 2.7183 \times 0.9048 = 2.4596$.)

$\text{PPL}_A \approx 3.32, \quad \text{PPL}_B \approx 2.46$
(If base-2 logarithms are used: $\text{PPL}_A = 2^{1.2} \approx 2.30$, $\text{PPL}_B = 2^{0.9} \approx 1.87$; the ranking is the same.)

(c) Which model performs better?

Model B performs better.

Justification:

1. Model B has the lower average cross-entropy ($0.9 < 1.2$) and the lower perplexity ($2.46 < 3.32$).
2. Average probability assigned to the correct next token (geometric mean) = 1/PPL: Model A $= 1/3.32 = 0.301$; Model B $= 1/2.46 = 0.407$. Model B gives about 35% more probability to the true tokens ($0.407/0.301 \approx 1.35 = e^{0.3}$).
3. Model A is as uncertain as choosing among about 3.3 tokens at each step, while Model B is as uncertain as choosing among about 2.5 tokens, so B is less "surprised" by the test data.
4. The comparison is fair because both models were evaluated on the same 100-token test sequence.

Q8. Multimodal LLMs and CLIP

Explain multimodal LLMs and the need for cross-modal learning. Explain how CLIP integrates image and text representations. System A: text in → text out. System B: text + image in → text out. Query: "What objects are present in this image, and what is the relationship between them?" with an image of a person riding a bicycle. (a) How each system handles the query. (b) Compare capabilities and limitations.

Part 1: Multimodal LLMs and cross-modal learning

Multimodal LLM (MLLM): A large language model that can take inputs from (and sometimes produce outputs in) more than one modality – text, images, audio, video – and reason over them jointly. Examples: GPT-4V/GPT-4o, Gemini, LLaVA, Flamingo, BLIP-2, Claude.

Typical architecture:

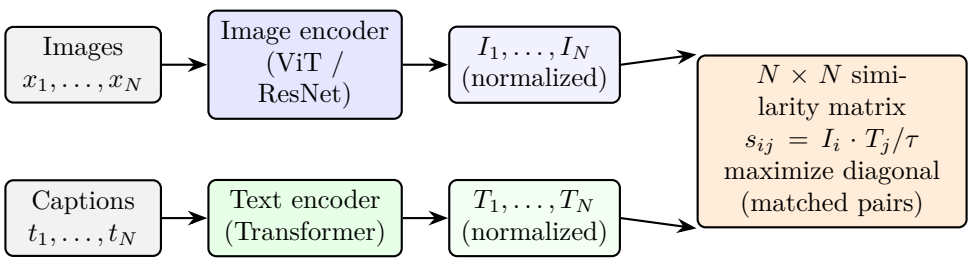
1. **Modality encoder:** e.g. a Vision Transformer (ViT) or CLIP image encoder that converts an image into a set of feature vectors (patch embeddings).
2. **Projection / connector:** A linear layer, MLP, Q-Former (BLIP-2) or cross-attention (Flamingo) that maps visual features into the LLM's token-embedding space ("visual tokens").
3. **LLM backbone:** The visual tokens are placed alongside text tokens; the LLM attends over both and generates the text answer.
4. **Training:** (i) alignment pre-training on image–caption pairs, (ii) visual instruction tuning on image-question-answer data.

Need for cross-modal learning:

- **Real-world information is multimodal:** documents with charts, photographs, medical scans, videos, screenshots – text alone cannot capture them.
- **Grounding:** Linking words to their visual referents (the word "bicycle" to the pixels of a bicycle) gives language a grounded meaning and reduces hallucination about visual content.
- **Complementary information:** One modality fills gaps in the other (e.g. an image shows spatial relations, text gives context and intent).
- **New tasks:** Visual question answering, image captioning, text-to-image retrieval, OCR and document understanding, accessibility (describing images for visually impaired users).
- **Shared representation:** A joint embedding space allows comparison and transfer between modalities, enabling zero-shot recognition.

Part 2: How CLIP integrates image and text representations

CLIP (Contrastive Language–Image Pre-training, OpenAI 2021) learns a *shared embedding space* for images and text using 400 million (image, caption) pairs from the internet.



1. **Two encoders:** an image encoder f (ViT or ResNet) and a text encoder g (Transformer). Each output is projected linearly into the same d-dimensional space and L2-normalized: $I_i = f(x_i)/\|f(x_i)\|$, $T_j = g(t_j)/\|g(t_j)\|$.
2. **Similarity:** cosine similarity $s_{ij} = I_i \cdot T_j/\tau$, where τ is a learned temperature.
3. **Contrastive objective (InfoNCE):** In a batch of N pairs, the N correct pairs (diagonal) should have high similarity and the $N^2 - N$ incorrect pairs low similarity. The symmetric loss is

$$\mathcal{L} = \frac{1}{2N} \sum_{i=1}^{N} \left[-\log \frac{e^{s_{ii}}}{\sum_{j=1}^{N} e^{s_{ij}}} \; - \; \log \frac{e^{s_{ii}}}{\sum_{j=1}^{N} e^{s_{ji}}} \right],$$

   i.e. cross-entropy for image→text plus text→image matching.
4. **Result – aligned space:** An image of a bicycle and the text "a photo of a bicycle" land close together. This enables:
   - *Zero-shot classification:* encode prompts "a photo of a {class}" for each class and pick the class whose text embedding is most similar to the image embedding.
   - *Image–text retrieval* in both directions.
   - *Vision backbone for MLLMs:* LLaVA, for example, uses the CLIP ViT encoder and a projection layer to feed language-aligned visual tokens into an LLM. Because CLIP features are already aligned with language, the connection to the LLM is easy to learn.
   - *Guidance for text-to-image generation* (e.g. in DALL-E 2 and Stable Diffusion's text encoder).

Part 3: Application to the given query

(a) How each system handles the query

System A (text-only):

- Its tokenizer and embedding layer accept only text. The image cannot be encoded, so it is either rejected by the interface or silently dropped; the model receives only the question text.
- Since the question refers to "this image" which the model cannot see, a well-behaved System A replies that it cannot view images and asks the user to describe the image in words.
- A poorly aligned System A may *hallucinate* a generic answer (invent objects) that has no connection to the actual image.
- If the user later types "a person is riding a bicycle", System A can reason over the text (objects: person, bicycle; relation: riding) – but this depends entirely on the user's description, not the image.

System B (text + image):

1. **Image encoding:** The image is split into patches and passed through a vision encoder (e.g. CLIP ViT), producing visual feature vectors.
2. **Projection:** The connector maps these features into visual tokens in the LLM embedding space.
3. **Fusion:** Visual tokens and the text question tokens are processed together; self-attention (or cross-attention) links words like "objects" and "relationship" to the relevant image regions.
4. **Generation:** The LLM generates a grounded answer, e.g.: "*The image contains a person and a bicycle (with possible background objects such as a road or trees). The person is riding the bicycle: seated on the saddle, hands on the handlebars, feet on the pedals. So the relationship is person – rides – bicycle; the bicycle supports and carries the person.*"

(b) Comparison of capabilities and limitations

Aspect	System A (text-only)	System B (multimodal)
Input modalities	Text only	Text and images
This query	Cannot answer from the image; needs a textual description	Answers directly: identifies person, bicycle and the "riding" relation
Object recognition	Not possible	Possible
Spatial / relational reasoning	Only from text descriptions	Directly from visual content (who is on what, positions, actions)
Grounding and hallucination	No visual grounding; high risk of made-up answers if it tries	Grounded in the image; lower risk, but can still hallucinate objects or misread fine details
Computational cost	Lower: no vision encoder, fewer tokens	Higher: vision encoder + hundreds of visual tokens per image; more memory and latency
Training data	Text corpora only	Needs large aligned image–text datasets and instruction data
Typical applications	Chat, writing, coding, text QA	Visual QA, captioning, document/chart/OCR understanding, accessibility, medical imaging assistance
Limitations	Blind to visual information	Limited image resolution, weaker at counting, small text and precise spatial relations; privacy concerns with images; adversarial images

Q9. GPT-3, In-Context Learning and Instruction Tuning

Explain the major developments in GPT-3 compared with earlier GPT models. Why can GPT-3 perform certain tasks without task-specific fine-tuning? Explain why instruction tuning improves the ability of GPT models to follow user instructions.

Part 1: Evolution and major developments in GPT-3

Feature	GPT-1 (2018)	GPT-2 (2019)	GPT-3 (2020)
Parameters	117M	1.5B	175B (> 100× GPT-2)
Layers / d / heads	12 / 768 / 12	48 / 1600 / 25	96 / 12288 / 96
Context length	512	1024	2048
Training data	BooksCorpus (∼5 GB)	WebText (∼40 GB)	∼300B tokens from filtered Common Crawl, WebText2, Books1/2, Wikipedia (∼570 GB filtered)
Usage paradigm	Pre-train + supervised fine-tuning per task	Zero-shot task transfer (weak)	In-context learning: zero-, one-, few-shot prompting without weight updates

Major developments in GPT-3:

1. **Massive scale:** 175 billion parameters, 96 layers, $d_{\text{model}} = 12288$, 96 heads, trained on about 300 billion tokens – validating the scaling laws.
2. **In-context learning (ICL):** The central contribution (paper title: "Language Models are Few-Shot Learners"). Tasks are specified entirely in the prompt:
   - *Zero-shot:* only a task description ("Translate English to French: cheese ⇒").
   - *One-shot:* description + one example.
   - *Few-shot:* description + several (10–100) examples within the context window.
3. **Architecture:** Same decoder-only design as GPT-2 (pre-normalization), with alternating dense and locally banded *sparse attention* patterns (as in the Sparse Transformer), and context doubled to 2048 tokens.
4. **Better data:** Common Crawl filtered with a quality classifier, fuzzy de-duplication, and a weighted mix in which high-quality sources (books, Wikipedia) are sampled more often.
5. **Performance:** Few-shot GPT-3 was competitive with fine-tuned models on several benchmarks (e.g. TriviaQA, LAMBADA), could do 2–3 digit arithmetic, word unscrambling, and generate news articles that humans found hard to distinguish from human-written ones.
6. **Paradigm shift:** From "one fine-tuned model per task" to "one general model, prompted for many tasks", delivered through an API.

Part 2: Why GPT-3 performs tasks without task-specific fine-tuning

1. **Meta-learning during pre-training:** Next-token prediction on a huge, diverse corpus implicitly exposes the model to countless tasks (translations, Q&A, lists, code, summaries) embedded in natural text. To predict well, the model must learn to recognize what task a piece of text is performing and continue it. The paper calls this the "outer loop" of learning (gradient descent during pre-training) and the "inner loop" (in-context learning at inference).

2. **In-context learning through attention:** At inference, the examples in the prompt act as conditioning information. The attention mechanism lets the model detect the input–output pattern in the examples and apply it to the new query, *without changing any weights*. Formally the model computes $P(y \mid \text{task description}, (x_1, y_1), \ldots, (x_k, y_k), x_{\text{query}})$.
3. **Scale:** ICL is an emergent ability – the gap between zero-shot and few-shot performance grows with model size and is largest for the 175B model; small models benefit little from examples.
4. **Universal text interface:** Every task can be expressed as text-in, text-out, so the same next-token predictor can perform classification, translation, QA, etc.
5. **Broad knowledge stored in parameters:** 175B parameters store extensive world and linguistic knowledge that can be retrieved by an appropriate prompt.

Part 3: Why instruction tuning improves instruction following

Instruction tuning is supervised fine-tuning of a pre-trained LLM on a large collection of tasks written as natural-language instructions paired with desired outputs, e.g. ("Summarize the following paragraph in one sentence: ...", summary). Examples: FLAN, T0, InstructGPT (SFT stage), Alpaca.

Why it helps:

1. **Fixes objective mismatch:** A base model is trained to *continue* text, not to *obey* it. Given "Write a poem about the sea", a base GPT-3 may continue with more similar instructions or an unrelated paragraph. Instruction tuning changes the learned behaviour to "given an instruction, produce a response that fulfils it".
2. **Generalization to unseen instructions:** Training on hundreds or thousands of diverse task types teaches the general skill of understanding instructions, so the model performs well zero-shot on new tasks it was never trained on (FLAN showed strong zero-shot gains on unseen task clusters).
3. **Less prompt engineering:** Users can state what they want directly, without carefully crafted few-shot examples.
4. **Response format and style:** The model learns to give complete, well-structured, concise answers, follow format constraints (lists, word limits, JSON), and use a conversational turn structure.
5. **Foundation for alignment:** Instruction tuning (SFT) is the first stage of RLHF; combined with RLHF (InstructGPT), outputs become more helpful, truthful and less toxic. The 1.3B InstructGPT was preferred by humans over the 175B GPT-3.
6. **Scales well:** Gains increase with the number of tasks and model size (Flan-PaLM), and including CoT examples in the instruction data also improves reasoning.

Q10. Deploying an LLM for a Customer-Support Chatbot

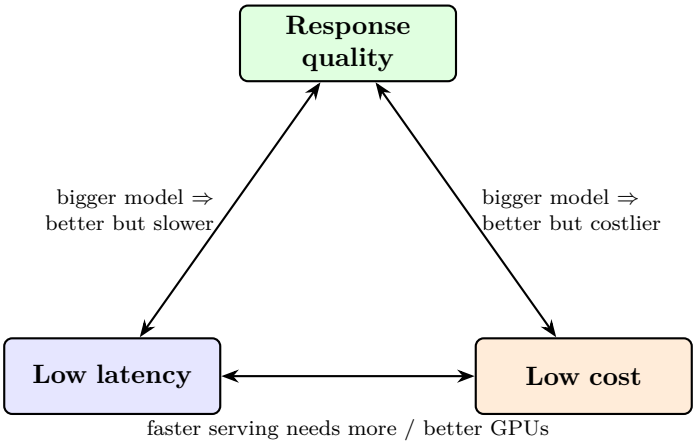
An organisation deploys an LLM customer-support chatbot receiving 1,000 queries per hour, requiring accurate responses with minimum delay. (a) Four challenges in deployment. (b) Trade-off between response quality, inference latency and computational cost. (c) Three measures to improve efficiency and reliability.

Load analysis: 1,000 queries/hour $= \dfrac{1000}{3600} \approx 0.28$ queries/second on average, i.e. about one query every 3.6 s. Traffic is not uniform, so peak load may be several times higher; the system must be sized for peaks, and each query may involve multiple turns and hundreds of generated tokens.

(a) Four challenges in deploying the LLM

1. **Hallucination and factual accuracy:** The LLM may confidently invent wrong information – non-existent policies, wrong prices, incorrect refund rules or order statuses. In customer support this damages trust and may cause legal or financial liability. The base model also does not know company-specific or up-to-date information.
2. **Inference latency and scalability:** Autoregressive generation produces one token at a time; a 300-token reply from a large model can take several seconds. Under peak traffic, requests queue up, increasing waiting time. Long conversation histories increase the prompt length and the KV-cache memory.
3. **High computational cost:** Large models need expensive GPUs (e.g. a 70B model needs ≈ 140 GB in FP16, i.e. multiple GPUs), or the organisation pays per token for an API. Cost grows with traffic, prompt length and output length.
4. **Data privacy and security:** Queries contain personal data (names, addresses, order and payment details). This must comply with data-protection laws (e.g. GDPR, India's DPDP Act). There are also risks of prompt injection or jailbreaks, leaking internal instructions or other customers' data.
5. **Other challenges (additional):** maintaining context over multi-turn conversations and context-window limits; consistency of brand tone; handling ambiguous, angry or out-of-scope queries and knowing when to hand over to a human agent; multilingual users; keeping knowledge updated; bias and toxic outputs; monitoring and evaluation of quality in production.

(b) Trade-off between response quality, inference latency and computational cost



The three goals pull against each other; improving one usually worsens at least one other:

1. **Quality vs latency:** Larger models (e.g. 70B) give more accurate, fluent and context-aware answers, but each token requires more computation, so time-to-first-token and time per token increase. Techniques that improve quality – retrieval (RAG), long context, chain-of-thought, self-verification – add more tokens and steps, increasing latency further. Smaller models respond faster but make more mistakes.
2. **Quality vs cost:** Inference cost is roughly proportional to (model size) × (input + output tokens). A large model needs more and costlier GPUs (or a higher API price per token). Using fewer/smaller GPUs, aggressive quantization (INT4) or shorter answers reduces cost but can lower accuracy.
3. **Latency vs cost:** Low latency at peak traffic requires more GPU replicas kept running (often under-utilised at off-peak hours), which raises cost. Large batching improves GPU utilisation and lowers cost per query, but each request may wait longer for its batch, increasing latency.
4. **Approximate quantitative view:** Generation time ≈ time-to-first-token + (number of output tokens × time per token). Time per token grows with model size, and memory needed ≈ parameters × bytes per parameter + KV cache. Halving precision (FP16 → INT8) halves weight memory and often speeds decoding with only small quality loss.
5. **Balancing for this chatbot:** The objective is the *smallest/cheapest configuration that meets the accuracy target and a latency target* (e.g. first token in under 1 s). Typical balance: a medium-size, fine-tuned model with RAG for accuracy; quantization and batching for cost; streaming responses so users see text immediately; routing only hard queries to a larger model.

(c) Three measures to improve efficiency and reliability

1. **Retrieval-Augmented Generation (RAG) with guardrails (reliability):** Store FAQs, product manuals, policies and order information in a vector database. For each query, retrieve the most relevant passages and include them in the prompt, instructing the model to answer only from this context and cite it. This grounds answers in up-to-date company knowledge, reduces hallucination, and avoids frequent re-training. Add input/output guardrails: PII masking, prompt-injection filtering, content moderation, and confidence checks that *escalate to a human agent* when the answer is uncertain or the query is sensitive (refunds, complaints).
2. **Model optimization – fine-tuning a smaller model, quantization and distillation (efficiency):** Fine-tune a smaller open model (e.g. 7B–13B) on the company's past support conversations using LoRA/QLoRA, or distil a large model into a small one. Apply INT8/INT4 quantization to reduce memory (e.g. 7B: 14 GB in FP16 → ≈ 3.5 GB in INT4) and increase speed, so the model fits on a single GPU with good domain accuracy.
3. **Efficient serving infrastructure – caching, batching and autoscaling (efficiency and reliability):**
   - *Response/semantic caching:* Many support questions repeat ("Where is my order?", "How do I reset my password?"). Cache answers for frequent or semantically similar queries and return them instantly without calling the LLM.
   - *Optimized inference engines:* vLLM/TensorRT-LLM/TGI with continuous batching, PagedAttention for the KV cache, prefix caching of the long system prompt, and streaming of tokens to the user.
   - *Query routing:* Simple intents handled by a small model or rules; only complex queries go to a larger model.
   - *Autoscaling and load balancing:* Add/remove GPU replicas according to traffic to handle peaks without paying for idle capacity; fallback to a backup model or human on failure.
4. **Continuous monitoring and evaluation (additional):** Log latency, cost per query, resolution rate, escalation rate and user feedback (thumbs up/down); regularly evaluate answers on a

test set; update the knowledge base and refine prompts or fine-tuning data based on failures.

End of solutions